Forensic Science International: Digital Investigation 39 (2021) 301271

Contents lists available at ScienceDirect

Forensic Science International: Digital Investigation

journal homepage: www.elsevier.com/locate/fsidi

File fingerprinting of the ZIP format for identifying and tracking provenance

Minji Um, Jaehyeok Han, Sangjin Lee*

Institute of Cyber Security & Privacy (ICSP), Korea University, 145 Anam-ro, Seongbuk-gu, Seoul, South Korea

ARTICLE INFO

Article history:
Received 16 June 2021
Received in revised form
24 August 2021
Accepted 30 August 2021
Available online 21 September 2021

Keywords:
ZIP file forensics
File fingerprints
User behavior
Compression and decompression characteristics

ABSTRACT

While the overall structure of ZIP files is defined, their detailed structure differs depending on the operating system and application creating the file. These characteristics are also affected by the environment in which the file was first created or later modified. Conversely, analyzing the structure of ZIP files allows the determination of the environment it was created in, and this can be the basis for determining where the file was created through analyzing and comparing the user's PC. In addition, the creation, modification, and access time values of decompressed files are set differently according to the application used for decompression and the structure of the ZIP file. ZIP files reflect not only the environment in which they are created but also the one in which they were decompressed. Thus, the ZIP files' detailed structures and characteristics should be analyzed forensically. In this paper, it is suggested that the environment of file creation and modification can be inferred by analyzing the detailed structure of a single file by file fingerprints, and the characteristics of decompression can be compared with the applications installed on the system.

© 2021 Elsevier Ltd. All rights reserved.

1. Introduction

Although the overall structure of a frequently used file type is defined, the detailed structure of the files created by each manufacturer and developer may differ. For example, OOXML (Office Open XML) documents have the same XML-based ZIP structural features, but the composition of that internal XML differs depending on the application that generates the document, whether MS Office or Libre Office. Since many file formats have a basic structure on which more detailed structures may be formed depending on the specific application, the environments in which files are created can be tracked by analyzing these structures.

ZIP files, which are compressed files for transport purposes or storage efficiency, are widely used, especially as they are provided as standard during operating system installation. While there is no superficial difference when using the ZIP file generated by various applications and operating systems, the detailed structure reveals differences, such as of encoding method or selective headers. It is forensically important to look at this detailed structure because the structure of ZIP files stores the time information of the compression target files. This allows one to infer the creation and modification environment through the structure of a compressed file, and the analysis of artifacts allows a more detailed determination of the source of the file. In terms of forensics, this is an important feature in that the source of a file can be used to hypothesize a user's behavior for a given event.

Differing structures of ZIP files depending on the creation environment and the features of the decompression applications affect the time value of the decompressed file. It means that both the creation environment and the decompression environment of the ZIP file can be inferred through the time information of the decompressed file. In addition, the application used for decompression can lead to reversal phenomena between the creation time and modification time of the decompressed folder. Comparing the time values of the folder and analyzing system artifacts allows a basis for the possibility of decompression to be presented.

The environment in which a file is created affects both the compression and decompression process. The creation environment can be determined from the structure of a ZIP file, furthermore, it can be checked through the time pattern values of the decompressed files and folders. Therefore, in this paper, we propose that the source of a ZIP file can be identified through a detailed structural analysis of the ZIP file and the characteristics of the decompressed file and folder.

* Corresponding author.
E-mail addresses: um2inji@korea.ac.kr (M. Um), one01h@korea.ac.kr (J. Han), sangjin@korea.ac.kr (S. Lee).

https://doi.org/10.1016/j.fsidi.2021.301271
2666-2817/© 2021 Elsevier Ltd. All rights reserved.

2. Background and related works

2.1. The structure of a ZIP file

The ZIP file has a signature '0 × 50 4B 03 04' and the overall structure is as in Fig. 1, with the Local File Header, Central Directory Header, and End of Central Directory(EOCD) (Katz, 2007). The Local File Header and Central Directory Header store information such as CRC32, the last modification time, the file name, and the compression method. The End of Central Directory exists in the last area of the compressed file, and stores the start position of the Central Directory Header and the number of compressed files and folders. Each header contains an extra field of variable length that differs depending on the file being compressed and the application used (see Figs. 2, 3 and 4).

Optional structures include the Data Descriptor and ZIP64 headers. The data descriptor facilitates data streaming and stores CRC32 and the sizes of the compressed and decompressed files. The ZIP64 End of Central Directory Record and ZIP64 End of Central Directory Locator, which are ZIP64 headers, are created depending on the size or number of files to be compressed. The ZIP64 EOCD Locator records the actual offset of the ZIP64 EOCD Record and the ZIP64 EOCD Record stores information such as the starting offset of the Central Directory and the number of compressed files.

2.2. Identifying the source of the file

The information constituting the file changes according to user behavior, or there is a difference in the structure of the file according to the creation environment. This is important from a forensics perspective because it allows us to identify the sources of files and reconstruct events. Existing studies identify the source of a file through the metadata inside a document file or identify the source through the differences in the structure of multimedia files in image or video format.

According to Joun et al. (2021), the metadata inside document files can be used to identify the sources of documents. In MS Word, whenever a document file is created, modified, deleted, or saved, a unique value, the RSID (Revision Identifier), is stored as metadata inside the file. By analyzing the RSID, it is possible to identify not only the history of content modification, deletion, or addition, but also whether the file itself or part of the content is copied, and even the type of document editing program used to create the file. In addition, Fu et al. (2011) have shown that it is possible to identify the source and copyright holder of an MS Office document by analyzing its Revision Identifier and attributes.

Mullan et al. (2019) have shown that source information, such as about device operating system and application information, can be identified by analyzing the JPEG header information taken from an iPhone. When analyzing changes in the number of entries per EXIF directory ('ExifIFD', 'IFDO', 'IFD1', 'GPS' and 'Maker Notes') over time, he found that they depend on the operating system version. In addition, the kinds of applications used can be identified by the characteristic quantization matrix found in the application. According to Gloe et al. (2014), the source of not only image files but also video files can be identified through their structure. Since the video format only defines essential functions, there are differences in detailed structure, such as container format and compression algorithms for audio and video depending on manufacturers and developers. Using these characteristics, it is possible to identify both the source of the file and the type of photographic device used and to know whether or not the video file was edited.

The ZIP file also stores information in the header about the creation environment, and the detailed structure differs depending on the application used to create the compressed file. Since the source of the file can be identified using these characteristics, a

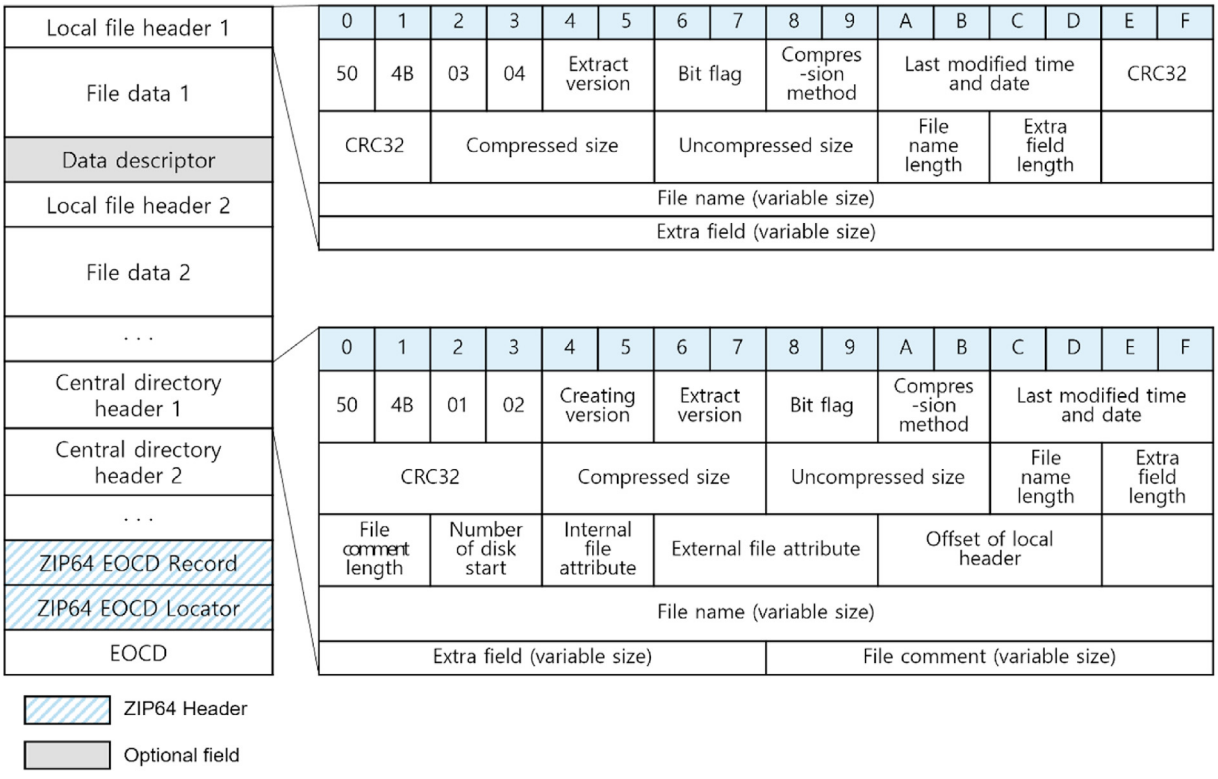


Fig. 1. Schematic structure of a ZIP file.

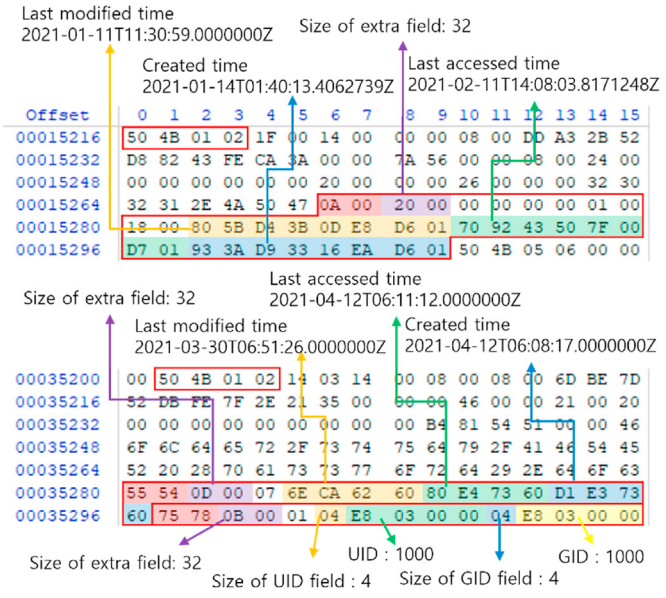


Fig. 2. NTFS (0 × 000A), Extended timestamps (0 × 5455), UNIX UID/GID (0 × 7875).

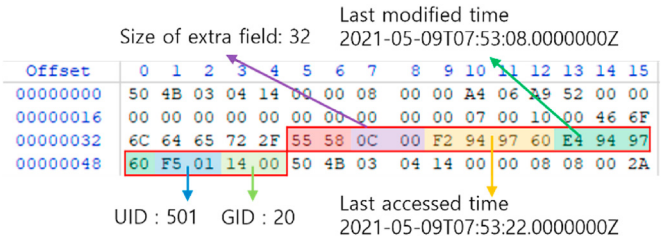


Fig. 3. Info-ZIP Unix extra field (0 × 5855).

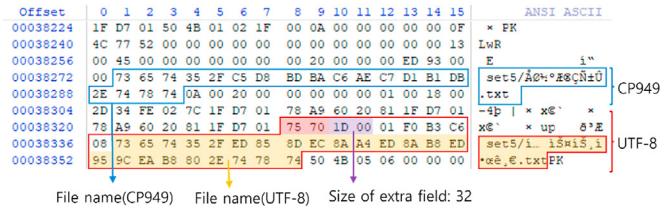


Fig. 4. Info-ZIP Unicode path extra field (0 × 7075), UTF-8, CP949.

detailed analysis of the structure of the ZIP file is required forensically.

3. Experiments for identifying file fingerprints

Even if the same object is compressed, the resulting ZIP files are created differently depending on the creation environment. We configured various system environments and then created a number of sample ZIP files as a dataset.[1] We then confirmed that the files contain identifiers that can provide information about their creation environments. The identifier differs depending on the type of operating system or the method (including the type of application) used by the user to compress the file. This section describes experimentally identifying the file fingerprints of the ZIP file that indicates the system environment.

The test operating systems were Windows 10 version 2004 (English and Korean edition), macOS 10.15, and Ubuntu 20.04 LTS. On Windows the files and folders were compressed with WinRAR v6.00, WinZip v25.0, 7-zip v19.00, Bandizip v6.29, and using *Compressed folder*, which is the Windows built-in function executable through a right-click, 'Send to' and the 'Compressed (zipped) folder' button. The *zip* command and *Compress* utility in macOS and Ubuntu were also used to create a ZIP file.

In addition, we installed Bandizip v7.0 and WinZip v8 on macOS (7-zip and WinRAR are not supported), and 7-zip (7zz) v21.01 was used on Ubuntu (Bandizip, WinRAR, and WinZip are not supported). When checking the differences in compression according to the encoding method used, files and folders with ASCII and Unicode file names were selected as compression targets. To check the compression method according to folder type, an empty folder or a folder with a subdirectory structure were selected as compression targets. ZIP files were also included to determine whether compressed files were recompressed. To initiate the compression process, in Windows, the quick menu was used, and in macOS, only *zip* was run in the terminal while the rest of the applications were run through the right-click menu. In Ubuntu, *zip* and 7-zip were run in the terminal, and only *Compress* was run through right-click menu.

- Usage of the Extra Field in the header
  - Is there a specific value in the Extra field?
  - What is the encoding method for expressing the file name?
- Methods of data processing for compression
  - Is the folder's local file header created?
  - Does the compression target contain ZIP files?
  - By what criteria was the compression order of the target files determined?
- Existence of system-generated data
  - Does the file name contain a specific string?
  - Is the data descriptor included?

3.1. Usage of the extra field in the header

The extra field is for expansion to store additional information for special needs or specific platforms. Earlier versions of the software worked to rarely use this field or skip it for stability. Currently, it is widely used by many software and its information is well-documented (Katz, 2007). In the *Extra Field*, each header should consist of *Header ID* (2 bytes), which indicates the type of data in the following data block, and *Data Size* (2 bytes). Header IDs of 0–31 are reserved for use by PKWARE and the remaining IDs have been used by third-party applications for proprietary usage. Thus, programs are allowed to store different types of information in the *Extra Field* of ZIP files.

3.1.1. Extended timestamps

A ZIP file contains a field (4 bytes) to store the last modification time of a compressed target in the local file header and central directory header. The time expression method is a 32-bit MS-DOS time format (Boyd and Forster, 2004). In general, the created, last modified, and last accessed timestamps information is provided by the file system. Due to the structure of ZIP files, the space available to contain this information is limited, allowing a time precision of up to 2 s. Therefore, the time information provided by the file system is stored in the *Extra Field*. For Windows, it is stored in an

[1] https://github.com/um2inji/PKZIP.

extra field that is created only in the central directory header (not in the local file header), and in macOS and Ubuntu, the time information can also be found in the extra field of the local file header.

The information stored in this extra field may include different content depending on the operating system that compresses the file. When a file is compressed in Windows, the header ID is 0 × 000A (NTFS), as shown in Table 1, and the data size is set to 32 bytes. In the data area, the creation, modification, and access times are stored in the 64-bit Windows FILETIME format (8 bytes) stored in the $STANDARD_INFORMATION of the file in the $MFT file. Experimentally, except for the ZIP file created using the built-in *Compressed folder* in Windows, whatever application was used for compression, an *Extra Field* with a header ID of 0 × 000A was created. Even when 7-zip is used in Ubuntu, the 0 × 000A extra field exists. Although not an NTFS file system, Ubuntu can express nanoseconds because there is an extra time field. Therefore, the modify, access, and change times checked through the 'stat' command of Ubuntu are stored in the 0 × 000A extra field.

For the *zip* and *Compress* of macOS or Ubuntu, when the ZIP file is compressed, 0 × 5455 (Extended timestamps, Table 2) that stores time information in 32-bit Unix time format (4 bytes), and 0 × 7875 (UNIX UID, Table 3) that stores UID (User Identifier) and GID (Group Identifier) are generated as the header ID. Depending on the compression method, the length of the 0 × 5455 extra field is stored as 0 × 13, 0 × 09, or 0 × 05. When the length of the extra field is 0 × 13, the last modified time, the last accessed time, and the change time are stored. When the length is 0 × 09, the last modified time and the last accessed time are stored, and in the case of 0 × 05, only the last modified time is stored. In macOS, when WinZip is used for compression, the extra field to store the time value is not created, but when Bandizip is used, 0 × 5855 (the Unix Extra Field, Table 4) is created as the header ID. The 0 × 5855 extra field in the local file header stores the last accessed time and last modified time in 32-bit Unix time format. And the header also stores UID and GID values, while the extra field in the central directory header stores only time values.

The creation environment can also be traced through a time zone analysis of the time information stored in the structure of ZIP files. The modified time value of the local file header or the central directory header stores time information reflecting the time zone value applied to the system that creates the ZIP file, and in the case of 0 × 000A, 0 × 5455, and 0 × 5855 in the extra field, it is saved based on UTC+0. As a result, in most cases where time information is stored in the extra field, the time zone can be determined by calculating the difference between the two time information values. For example, if a system that creates a compressed file has a time zone of UTC-5 (EST) and an application that references the time value stored in the header on a system as UTC+9 (KST), the files or folders that are decompressed can be as different in terms of the timestamp as the time zone difference set on the system that generated the compressed file.

The timestamp from the extra field can be decoded as generic format *YYYY − MM − DD hh*: *mm*: *ss.fffffff*, especially where different features exist depending on the compressed file, it is expressed in nanoseconds. WinZip has the characteristic that only a part of it is saved as 0, such as 'fff0000', and the extra field of the file compressed by the rest of the application is set to the same as that saved in *$STANDARD_INFORMATION*.

3.1.2. UID and GID

In order to identify the system environment in which the file was created or modified, the ZIP file by Bandizip, *Compress*, *zip* on macOS, *Compress* and *zip* on Ubuntu can be used the UID and GID using extra field header ID 0 × 5855. Also, they were included in the structure of the ZIP file. The default UID and GID are 501 (0 × 1F5) and 20 (0 × 14) in macOS respectively. Meanwhile, both are 1000 (0x3E8) in Ubuntu.

When an account is added in macOS, the UID is increased by 1 from the default value, and the GID remains the same. In the case of Ubuntu, both values of the UID and GID are increased by 1, respectively. However, the value of the UID and GID can be arbitrarily modified by users. When a ZIP file is created in a new account, the structure of the ZIP file stores the UID and GID of that account. Through the structure of ZIP file, it is possible to figure out the account and ID values (UID and GID) used while compressing the ZIP file. The ID values stored in the structure of ZIP files should then be compared with the account information on the PC of the actual investigation target.

3.1.3. Language encoding flag (EFS)

The local file header and the central directory header have a field to store the file name. This value stores the language type of the string that makes up the file name or the language encoding associated with the system setting. And a header ID with a value of 0 × 7075 is created in the extra field for the purpose of saving the file name in Unicode.

Encoding is used as a way to express the languages used in different countries. If decoding is performed with a method other than that used by the encoding method, the encoded values cannot be properly checked. The same issue can occur with ZIP file analysis as for an e-mail analysis (Chhabra and Bajwa, 2015) (Moore, 1996), HTML (Hypertext Markup Language) analysis (Ishida, 2014), etc. These encoding problems can develop when using different language systems. To solve these problems, the file name should be stored in the structure of the ZIP file encoded in UTF-8, a method supported by many systems. In other words, the file name in the extra field is saved in UTF-8, which is the default method of encoding Unicode, and 0 × 7075, which means 'Unicode Path Extra Field' (Table 5), is recorded in the extra field as a flag (EFS) indicating that the file has been saved in Unicode.

The character encoding of the file name can also be used to estimate the system where the ZIP file was created. When Unicode is included in the compressed file name, it is encoded in a manner supported by language packs for Windows. For example, when Korean is included in the file name, CP949 (Microsoft Windows code page for Korean, Code Page 949, Windows-949) is used. The extra field containing the 0 × 7075 header ID has a file name including Unicode and was found only when compression was performed through WinRAR (Windows_winrar), WinZip (Windows_winzip), or Bandizip (Windows_bandi). Regardless of Language packs, this header was not created when English-only file names were used. The file name field in the local file header or central directory header was encoded in UTF-8 in the English Version of Windows and CP949 in the Korean Version of Windows. In fact, in the English version of Windows, compression using the *Compressed folder* function was not supported for files containing Unicode.

Table 1
Structure of NTFS header ID: 0 × 000A.

Offset	Size	Description
0	2	0 × 000A (Fixed value)
2	2	Size of extra field
4	4	Reserved
8	2	0 × 01 00(tag of attribute)
10	2	Size of attribute
12	8	Last modified time
20	8	Last accessed time
28	8	Created time

Table 2
Structure of extended timestamp: 0 × 5455.

Offset	Size	Description
0	2	0 × 5455 (Fixed value)
2	2	Size of extra field
4	1	Info bits - bit0: Last modified timestamp - bit1: Last accessed timestamp - bit2: Changed timestamp
5	4	Last modified time(optional)
9	4	Last accessed time(optional)
13	4	Change time(optional)

Table 3
Structure of UNIX UID/GID: 0 × 7875.

Offset	Size	Description
0	2	0 × 7875 (Fixed value)
2	2	Size of extra field
4	1	Version (currently 1)
5	1	Size of UID field
6	Variable	UID for this entry
-	1	Size of GID field
-	Variable	GID for this entry

Table 4
Structure of Unix extra field header ID: 0 × 5855.

Offset	Size	Description
0	2	0 × 5855 (Fixed value)
2	2	Size of extra field
4	4	Last accessed time
8	4	Last modified time
12	2	UID for this entry(optional)
14	2	GID for this entry(optional)

Additionally, a file name encoding process of decompressed ZIP files differs between Windows and macOS, especially with the Korean file name. Windows uses NFC (Normalization Form Canonical Composition) encoding method, which uses Korean syllable units. However, in macOS, the file name consists of Korean phoneme units because of NFD (Normalization Form Canonical Decomposition) encoding method. Also, the name of the file encoded by the NFD method can remain even when the file is moved and stored in a Windows-based computer.

Extra Field can be created in a local file header or a central directory header, and the order of header IDs in this field can be utilized as an identifiable characteristic. For example, in the central directory header, both 0 × 000A (Extended timestamp) and 0 × 7075 (EFS) can be created as header ID values, and the storage order of these values may differ depending on the environment in which the ZIP file was created. When files were created using Bandizip (Windows_bandi) or WinRAR (Windows_winrar) in Windows, data was created in the order of 0 × 000A-0x7075, but when using WinZip(Windows_winzip), data was saved in the order of 0 × 7075-0x000A.

3.2. Methods of data processing to compress

3.2.1. Handling folders

A folder (or directory) is a conceptual space used to classify files. When a folder is included in a compression target, a local file header and a central directory header for that folder may or may not be generated. Even if a folder's header does not exist directly, it can be decompressed through the folder name that exists in the file name field in the header.

In Windows, there are three ways to handle folders. Using the structure in Fig. 5 as an example, in the first way the header of the root folder ('set1') and the header of the subfolder ('Study') exist. Accordingly, Fig. 6 shows that the header of the root folder exists and the root folder is recorded in the name filed of the file in the root folder. This case was confirmed to occur when compression was performed through WinRAR, WinZip, 7-zip, or Bandizip. In the second way, there is no header for the root folder, but the information exists on the path of the file name field in the header, and the header for the subfolder, 'Study' does exist. In this case, it was possible to check when compression was performed through the *Compressed folder* of Windows. In the final way, in the case of empty folders being compressed, all applications generate the local file header and the central directory header.

3.2.2. Double zipping: compression for a ZIP file

ZIP files compress each file individually. If a ZIP file that has already been compressed sufficiently to no longer need further compression is included in the compression target, recompression is not performed for efficiency reasons. When not recompressed, the original structure of the compressed ZIP file is maintained, but when recompressed, that structure is lost.

For WinZip, 7-zip(Windows and Ubuntu), and Bandizip(Windows and macOS), recompression of the ZIP file included in the compression target was found in some cases. Recompression was performed when a low-level compressed ZIP file was included. However, no further compression was performed when a ZIP file having a high-level compression. Note that both features may exist because whether to recompress depends on the compression rate of the ZIP file included in the compression target.

Applications that do not recompress regardless of the compression rate of the ZIP file included in the compression target were WinRAR and the *Compressed folder* of Windows. Both programs identify ZIP files based on their extension. For example, when we simply changed the extension of a jpg file to a zip and then compressed it with the WinRAR or *Compressed folder* of Windows, we could see that the binary copy of the data of the jpg was stored in the compressed file. In other words, both applications identify ZIP files just based on their extension and then do not compress.

3.2.3. Order of files in an archive

In this paper, the compression order means the order of the local file header and the central directory header. When the folders are compressed by using the above Windows applications, they are compressed in order of number(with ascending order), English, and Korean(each by alphabetical order). The same situation happens while creating ZIP files no matter what the folder sorting criteria are, such as name, modified date, type, and size, etc. In some cases, applications can be distinguished by the order of the files in a ZIP file. When performing file-by-file compression through WinRAR on Windows, it can be compressed in a different order compared with

Table 5
Structure of Info-ZIP unicode path extra field: 0 × 7075.

Offset	Size	Description
0	2	0 × 7075 (Fixed value)
2	2	Size
4	1	Version
5	4	CRC32 of file name
9	variable	Unicode file name

the priority of folder-unit compression.

3.3. Existence of system-generated data

3.3.1. The '__MACOSX' folder on macOS

A file created on macOS consists of data forks that store the actual data part of the file and resource forks that store the metadata and application resources. The resource forks are included in a '__MACOSX' folder when creating the ZIP file. Although this folder is hidden on a PC using macOS, it can be viewed on a PC using another system. In the case of macOS, when compression was performed through *Compress*, the '__MACOSX' folder was created, and information about the creation environment of the ZIP file could be obtained through '__MACOSX' folder. However, when using *zip*, the folder was not created. The operating system of the compression environment can be identified through the system information included when the ZIP file is created, and the user application can be identified according to the internal structure. This property is meaningful in that it is possible to obtain information about creation environment by analyzing only the specific file, even without a system.

3.3.2. Data descriptor

In macOS and Ubuntu, a data descriptor was not created when compressing through *zip*, but when compressing through *Compress*, the data descriptor with the '0 × 50 4B 07 08' signature was created by default. In the case of the latter, the field storing CRC32 and the decompression size in the local file header were filled with NULL, and the value was stored in the area storing the file size after compression.

3.4. Summary

The structure of the ZIP file reflects the characteristics of the operating system and applications. Therefore, if ZIP files are created by the same application but on different operating systems, they will have different file fingerprints. For example, if the ZIP files are created by the same application, the created extra field would be the same, but the operating system can be distinguished by the creating version in the central directory header. Additionally, while Windows and Ubuntu use the NFC UTF-8 method, macOS uses the NFD UTF-8 method so the hexadecimal values for the same Korean alphabet letters differ. Since file fingerprints are created by combining the characteristics of operating systems and applications, the above source characteristics of the ZIP file should be investigated.

Table 6 shows the characteristics of ZIP files created by third-party applications, including the *Compressed folder* on Windows. Table 7 shows the characteristics of files created through *zip*, *Compress*, and third party applications on macOS and Ubuntu. Header ID means the ID value of the extra field that exists when the ZIP file is created. The timestamp storage method in the extra field reflects the NTFS difference in time precision of 100 ns for each compression program. The encoding method refers to the method to encode the file name field's value of the header. The header of a folder refers to whether a header is generated when a folder with a subdirectory structure or an empty folder in the compression target is compressed. Root folder indicates that whether headers are generated or not and also whether a path in the file name field of the headers includes the root folder or not when the root folder is compressed. Double zipping, refers to whether or not ZIP files are recompressed when they exist in the compression target. The compression order refers to the order in the header of the compressed file.

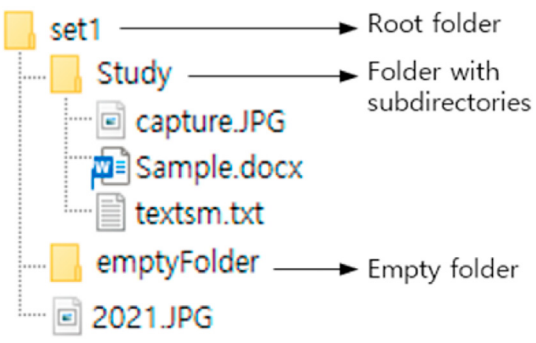


Fig. 5. An example directory structure in the experiment.

Table 7 presents the results of examining the types of extra fields that exist in the local file header and the central directory header, and the length of the 0 × 5455 extra field. The table also includes the encoding method, whether the ZIP file is recompressed, and the result of the existence of the '__MACOSX' folder and data descriptor. However, it should be noted that even for the same application, the characteristics may vary depending on which compression method is chosen in the menu.

4. Forensic analytical technique to track provenance

4.1. Development of an automated classifier

We developed the automated classifier to identify the system environment in which a ZIP file was created by combining the characteristics of the operating system and the application through structural analysis of the ZIP file. We could assume the case where a ZIP file has been modified or recompressed on a different system from the one in which it was created. To investigate this possibility, we classified file fingerprint types into reliable (Algorithm 1 and 2 of Fig. 8) or target-dependent characteristics (Algorithm 3 of Fig. 8).

When a ZIP file is entered into the automated classifier as in Fig. 7, the first thing to check is whether any robust and reliable characteristics exist. The automated classifier first checks for the existence of the data descriptors and the types of extra fields in each header. Next, the information about the operating system on which the ZIP file was created is checked through the creating version present in the central directory header. If the source of the ZIP file can be identified by combining the results of the above processes, the ZIP file's creation environment can be determined. If not, other characteristics such as double zipping, the existence of the '__MACOSX' folder, and the form of the nanosecond section of the timestamp are additionally checked. In the case of ZIP files created by operating systems or applications not covered in this paper, feedback and support for the individual file is required.

Some characteristics depend on the compressing-target file rather than the creation environment. In other words, the fingerprints reflective of an operating system or application different from the actual creation environment may exist in the structure of ZIP file. For example, the $STANDARD_INFORMATION of the compressed file in the $MFT file determines the values of the 0 × 000A extra field. If the form of the nanoseconds stored in $STANDARD_INFORMATION of the compressed file is 'fff0000', the values of the 0 × 000A extra field are the same for a specific application. These characteristics are considered of low reliability for identifying the source of the ZIP file, so further analysis of system artifacts is suggested.

We used the automated classifier to identify the source of the ZIP files stored on three PCs with Windows. Only *Compressed folder* had installed for the PC1. And, in the PC2, 7-zip, Bandizip, and

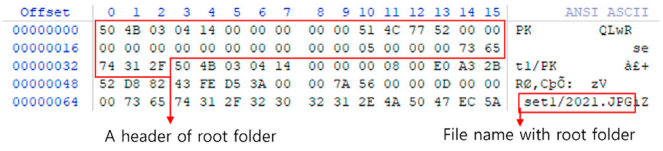


Fig. 6. Handling folders containing files and folders.

Compressed folder had installed. Last, the PC3 had 7-zip and *Compressed folder*. Fig. 9 shows the candidates of applications creating ZIP files and the number of ZIP files based on fingerprints. If one ZIP file has the characteristics of multiple applications, all the applications are included as candidates. As a result of the automated classifier, many ZIP files stored on PC1 had various fingerprints. In PC2, there were many ZIP files created through specific operating systems and applications, and in PC3, ZIP files that are compressed with 7-zip existed. In other words, in PC2 and PC3, it was confirmed that there are many ZIP files created through a specific application depending on the user's usage habits. However, in the public computer, it was confirmed that there are various distributions because there are ZIP files with various fingerprints like PC1. Based on these results, it is possible to determine that the ZIP files in PC2 and PC3 are both created locally or imported from outside, while most of the ZIP files in PC1, the public computer, are highly likely to have been imported from outside. Therefore, the source of the ZIP file can be identified by comparing the characteristics of the ZIP file stored in each PC with the operating system and applications installed in the PC.

After analyzing the characteristics of a single file, a further analysis of system artifacts can be performed to understand the specific creation environment of a file.

4.2. Supplementary experiment for changes in a given ZIP file

In general, ZIP files are created for a file or folder selected by the user, but in some cases, the file is appended to an already existing ZIP file, or only part of an existing zip file is deleted or modified. In fact, most applications support adding, deleting, and modifying existing ZIP files without decompressing them. Thus, we conducted an experiment to add, delete, and modify individual files in an existing ZIP file through an application and found the following two cases.

In the first case, the file fingerprints of only those files changed to reflect the characteristics associated with the zip application making the change (SW2). As shown in Fig. 10, when a ZIP file is compressed with SW1, (a) adding or (c) modifying a file using SW2 causes changes to the header of only the added or modified file, reflecting the characteristics associated with SW2. When (b) deleting a compressed file in the ZIP archive, only the header of that file is deleted. In this case, multiple systems' characteristics can exist in a single ZIP file.

In the second case, (d) when adding, deleting, or modifying an individual ZIP file, the fingerprints associated with the altering application are reflected not only in the header of the changed file but also in the headers of the unchanged files. For example, when an individual compressed file in the ZIP file is deleted, the header of that file is deleted, and the existing extra field is deleted or added to the remaining files' headers. This means that a change has occurred to the characteristics of the existing compressed file(s). Thus, checking whether file fingerprints with different characteristics exist in the structure of ZIP file is required.

Although, in general, one ZIP file has the fingerprints associated with one particular system, a detailed structural analysis is required because one or more fingerprints could exist on the ZIP file. If the file fingerprints of multiple systems coexist in a ZIP file, it is apparent that the ZIP file has been modified in two or more environments. The above analysis method is useful in that the modification history of the ZIP file can be determined by checking the structure of the ZIP file only and without having to analyze system artifacts.

4.3. Differences in decompression results for a given ZIP file

ZIP files have different fingerprints depending on the operating system or application, but even the result of decompressing a given ZIP file may yield different information from the state of the file before compression. For example, the time information (MACB - modification time (M), last access time (A), and MFT entry modification time (C), creation time (B) in the MFT entry) of the decompressed file can be recorded as a compressed time point or a decompressed time point. The reason is that the processing method of the Header ID in the extra field differs depending on the decompression application. When WinRAR decompresses a ZIP file possessing an extra field with a Header ID of 0 × 000A, it sets the creation and modification time values of the decompressed file as the decompression time and the values stored in the extra field, respectively. On the other hand, WinZip sets the creation time, modification time, and access time for the same ZIP file to the values that exist in the extra field so that whether or not reversal occurs depends on the time information stored in the extra field. In addition, it was confirmed by experiment that the existence of a folder header and the existence of a subdirectory structure (an empty folder or a folder with subdirectory structure) affect the setting of the time information of the decompressed file or folder following Table 8. At this time, decompression was performed in the right-click menu, and some options related to timestamp did not exist.

Therefore, even if the given ZIP file is the same, the result of decompression differs depending on the decompression

Table 6
File fingerprints of the ZIP format on the Windows.

Identifier		WinRAR	WinZip	7-zip	Bandizip	Compressed folder
Header ID (Ref. 3.1.1)	Local Extra Field	EN: N/A KR: 0 × 7075	EN: N/A KR: 0 × 7075	N/A	EN: N/A KR: 0 × 7075	N/A
	Central Extra Field	EN: 0 × 000A KR: 0 × 000A + 0 × 7075	EN: 0 × 000A KR: 0 × 7075 + 0 × 000A	0 × 000A	EN: 0 × 000A KR: 0 × 000A + 0 × 7075	N/A
Timestamp(nanoseconds) on the 0 × 000A extra field (Ref. 3.1.1)		ffffff	fff0000	ffffff	ffffff	N/A
Encoding Method (Ref. 3.1.3)		EN: UTF-8 KR: CP949	EN: UTF-8 KR: CP949	EN: UTF-8 KR: CP949	EN: UTF-8 KR: CP949	EN: UTF-8 KR: CP949
Header of a folder (Ref. 3.2.1)		Exist	Exist	Exist	Exist	Exist
Root folder (Ref. 3.2.1)	Header	Exist	Exist	Exist	Exist	Non-exist
	File name field	Exist	Exist	Exist	Exist	Exist
Double Zipping (Ref. 3.2.2)		X	O	O	O	X
Order of files in an archive (Ref. 3.2.3)		Alphabetical order	Alphabetical order	Alphabetical order	Alphabetical order	Alphabetical order

Table 7
File fingerprints of the ZIP format on the macOS and the Ubuntu.

Identifier		macOS				Ubuntu		
		WinZip	Bandizip	Compress	zip	7-zip	Compress	zip
Header ID (Ref. 3.1.1)	Local Extra Field	N/A	0 × 5855	0 × 5455 0 × 7875	0 × 5455 0 × 7875	N/A	0 × 5455 0 × 7875	0 × 5455 0 × 7875
	Central Extra Field	N/A	0 × 5855	0 × 5455 0 × 7875	0 × 5455 0 × 7875	0 × 000A	0 × 5455 0 × 7875	0 × 5455 0 × 7875
Length of 0 × 5455 Extra Field (Ref. 3.1.1)	Local File Header	N/A	N/A	13 bytes	9 bytes	N/A	13 bytes	9 bytes
	Central directory Header	N/A	N/A	13 bytes	5 bytes	N/A	13 bytes	5 bytes
Encoding (Ref. 3.1.3)		UTF-8	UTF-8	UTF-8	UTF-8	UTF-8	UTF-8	UTF-8
Double Zipping (Ref. 3.2.2)		O	O	O	X	O	O	X
OS folder (Ref. 3.3.1)		N/A	N/A	Exist	N/A	N/A	N/A	N/A
Data descriptor (Ref. 3.3.2)		N/A	N/A	Exist	N/A	N/A	Exist	N/A

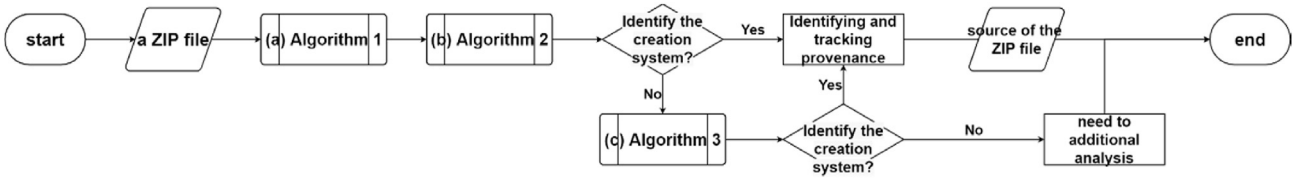


Fig. 7. Flowchart of the automated classifier.

Algorithm 1. Classifier with high weight - SW

```
1:  procedure estimate program_1(a ZIP file)
2:      result = []
3:      if exists Data descriptor then      → 3.3.2
4:          result.append('Data descriptor')
5:      if extra field is N/A then          → 3.1
6:          result.append('Extra N/A')
7:      else if extra field is 0x7075 then
8:          result.append('Exrta 0x7075')
9:      else if extra field is 0x000A then
10:         result.append('Extra 0x000A')
11:     else if extra field is 0x5855 then
12:         result.append('Extra 0x5855')
13:     else if extra field is 0x5455 and length then
14:         result.append('Extra 0x5455', length)
15:     else if extra field is 0x7875 then
16:         result.append('Extra 0x7875')
17:     return result
```

Algorithm 2. Classifier with high weight - OS

```
1:  procedure estimate program_2(a ZIP file)
2:      result = []
3:      if Creating version is 0x00 then
4:          result.append('Windows')
5:      else if Creating version is 0x03 then
6:          result.append('macOS')
7:          result.append('Ubuntu')
8:      return result
```

Algorithm 3. Subsidiary classifier with low weight

```
1:  procedure estimate program 3(a ZIP file)
2:      result = []
3:      if exists a double zipping then     → 3.2.2
4:          result.append('dobule zipping')
5:      if exists '__MACOSX' folder then    → 3.3.1
6:          result.append('__MACOSX')
7:      if form of nanoseconds is .fff0000 then → 3.1.1
8:          result.append('fff0000')
9:      else if form of nanoseconds is .fffffff then
10:         result.append('fffffff')
11:     else if form of nanoseconds is .0000000 then
12:         result.append('0000000')
13:     if Unicode form is NFC(UTF-8) then  → 3.1.3
14:         result.append('Ubuntu')
15:     else if Unicode form is NFD(UTF-8) then
16:         result.append('macOS')
17:     else if encoding method is CP949 then
18:         result.append('Windows')
19:     if exist folders' headers then      → 3.2.1
20:         result.append('folder header')
21:     if exist UID and GID then           → 3.1.2
22:         result.append(UID, GID)
23:     if alphabetical order then          → 3.2.3
24:         result.append('alphabetical')
25:     return result
```

Fig. 8. Algorithms in the process of the flowchart.

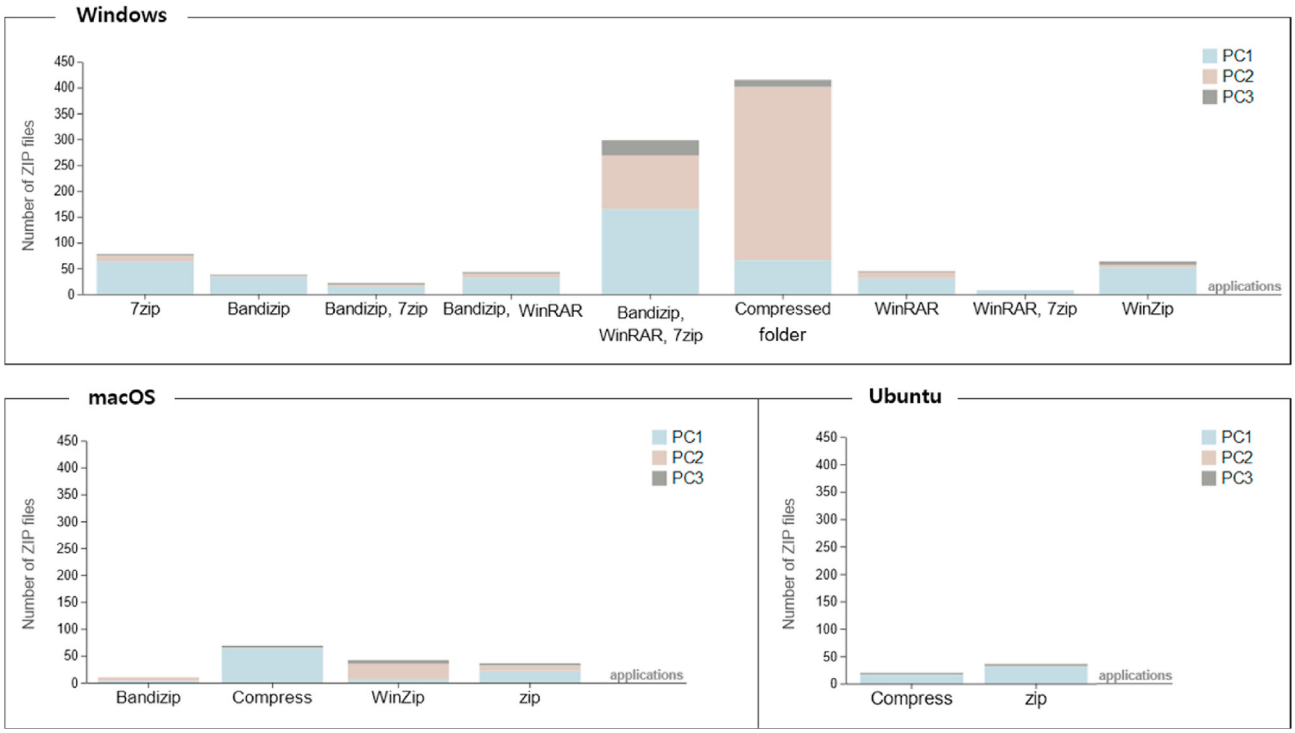


Fig. 9. Results of the provenance tracking for ZIP files stored on the Data set of the experiments (Computers classified into three types).

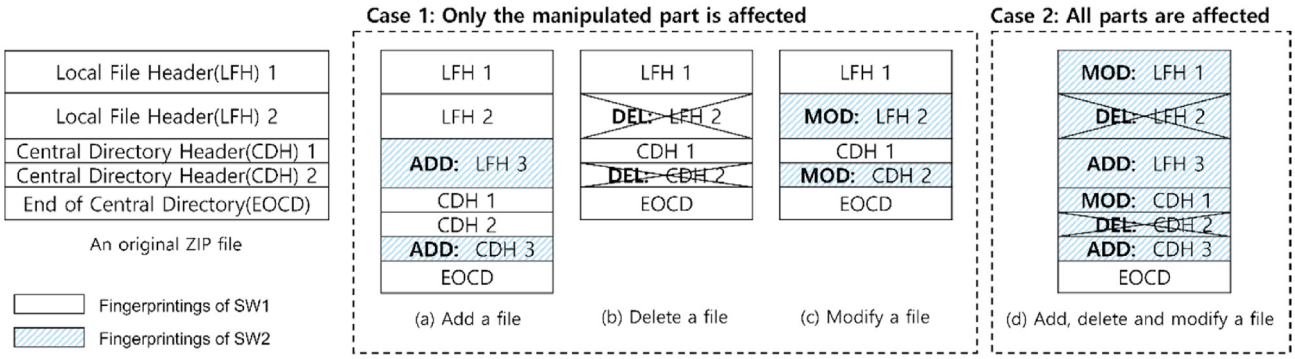


Fig. 10. Changes in ZIP files when added, deleted, and modified by different software: Experimental results.

Table 8
Timestamp records when a given ZIP file was decompressed on Windows (Experimental results). *R*, *H*, and *E* mean decompressed real time, last modified time in the header, and timestamps with HeaderID in the extra field respectively.

Give ZIP file info.			Decompressor/Timestamps(BMAC in order)				
Attribute	State	HeaderID	WinRAR	WinZip	7-zip	Bandizip	Compressed folder
File	–	None	RHRR	HHHR	RHRR	RHRR	HHHR
		0 × 000A	RERR	EEER	EEER	EEER	HHHR
		0 × 5455	RERR	HHHR	RERR	RHRR	HHHR
		0 × 5855	RHRR	HHHR	REER	RHRR	HHHR
Folder	No header	N/A	RRRR	RRRR	RRRR	RRRR	RRRR
	Contains file(s)	None	RHRR	HRRR	RHRR	RRRR	RRRR
		0 × 000A	RERR	ERRR	EEER	ERRR	RRRR
		0 × 5455	RERR	HRRR	RERR	RRRR	RRRR
		0 × 5855	RHRR	HRRR	REER	RRRR	RRRR
	Empty	None	RHRR	HHHR	RHRR	RHRR	RHRR
		0 × 000A	RERR	EEER	EEER	EEER	RHRR
		0 × 5455	RERR	HHHR	RERR	RHRR	RHRR
		0 × 5855	RHRR	HHHR	REER	RHRR	RHRR

application and different states of the ZIP file, such as extra fields and differences in internal structure, even when using the same decompression application. In particular, the investigation of the time-reversal phenomenon of a file or folder cannot be judged by merely considering a general situation. We confirmed through experiments that a reversal may occur between the creation time and modification time of a decompressed folder, and also between the time of a parent folder and the time of a subfolder. Thus the analysis of a given ZIP file and the analysis of the artifacts (or logs) of the system environment in which that file was stored must be accurately investigated together.

5. Conclusion and future work

We have looked at the characteristics of ZIP files created through various actions on Windows, macOS, and Ubuntu. It was possible to determine which operating system had created or modified a ZIP file through the detailed file characteristics. In particular, in the case of Windows, it was possible to determine the language pack and the application used for compression. The file characteristics are the file-level analysis result, to counter anti-forensic techniques, one should carefully determine the source of the file in parallel with traditional investigation techniques. If the artifacts of a system and a single file are analyzed together, the source of the file can be determined in detail. If the PC under investigation and the environment in which a suspected file has been created are inconsistent, it suggests that another PC related to the case exists. Suppose that the PC's system and the operating system suspected of generating the ZIP file related to the case do not match, or that the application that is thought to have been used for creation is not present. Then it can be assumed that either the system has been reinitialized or the file has been introduced via the Internet or external storage devices.

OOXML and JAR(Java Archive) formats are typical examples of sharing the same structure with a ZIP file format. In particular, OOXML can be created through various applications such as MS Office or LibreOffice. Depending on the applications, each OOXML has a different structure with the existence of a data descriptor, an extra field in the local file header, etc. By using these structural differences, it is possible to determine the application that created the file. These characteristics reveal a possibility of identifying the source of document files. Therefore, a further study will proceed to analyze a method of investigating the source of document files.

It is important to look at the detailed structure of ZIP files as ‘fingerprints’, because the detailed characteristics of the ZIP files identified for each application differ. However, since characteristics may also differ depending on the operating system and the version of the application used, continuous updates are required for the analysis process. In addition, there are several ways to compress even using one application. The characteristics of the compressed file may vary depending on the compression method. For example, the existence of the root folder's header depends on the compression method while using the Bandizip application. Therefore, to extend the fingerprints database for covering various compression applications, further analysis is required.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

Boyd, C., Forster, P., 2004. Time and date issues in forensic computing—a case study. Digit. Invest. 1 (1), 18–23.
Chhabra, G.S., Bajwa, D.S., 2015. Review of e-mail system, security protocols and email forensics. Int. J. Comput. Sci. Commun. Netw. 5 (3), 201–211.
Fu, Z., Sun, X., Liu, Y., Li, B., 2011. Forensic investigation of ooxml format documents. Digit. Invest. 8 (1), 48–55.
Gloe, T., Fischer, A., Kirchner, M., 2014. Forensic analysis of video file formats. Digit. Invest. 11, S68–S76.
Ishida, R., Jun 2014. Authoring HTML: language declarations. W3C note, W3C. https://www.w3.org/TR/2014/NOTE-i18n-html-tech-lang-20140603/.
Joun, J., Chung, H., Park, J., Lee, S., 2021. Relevance analysis using revision identifier in ms word. J. Forensic Sci. 66 (1), 323–335.
Katz, P., 2007. Appnote. txt—. zip file format specification. https://pkware.cachefly.net/webdocs/casestudies/APPNOTE.TXT.
Moore, K., Nov 1996. MIME (Multipurpose Internet mail extensions) Part Three: Message header extensions for Non-ASCII Text. RFC 2047. https://doi.org/10.17487/RFC2047. https://rfc-editor.org/rfc/rfc2047.txt.
Mullan, P., Riess, C., Freiling, F., 2019. Forensic source identification using jpeg image headers: the case of smartphones. Digit. Invest. 28, S68–S76.